

Table of Contents

1. API & HTTP Fundamentals	1
HTTP Request Anatomy	2
Path Variable vs Query Parameter	2
HTTP Headers (Metadata)	2
HTTP Methods	2
HTTP Status Codes	2
2. Spring Boot REST API Implementation	3
Standard Layered Architecture	3
Key Dependencies	3
Serialization vs Deserialization (Parsing)	3
DTO (Data Transfer Object)	3
Complete CRUD Controller Structure	3
ResponseEntity	4
Validation & Exception Handling	4
3. Consuming External APIs & LLMs	4
RestClient (Spring Boot 3.2+)	4
LLM API Integration Architecture	5
4. REST vs GraphQL	5
REST Principles	5
GraphQL Basics	5
Spring Boot GraphQL Annotations	5
Comparison Table	6
5. Spring Boot Annotations Cheat Sheet	6
6. End-to-End Mental Model (Request → Response)	6
7. Interview Prep Checklist & Questions	6
HTTP & API Concepts	7
Spring Boot REST	7
GraphQL	8

1. API & HTTP Fundamentals

- **What is an API?** Application Programming Interface. A defined interface allowing applications to communicate. (e.g., Frontend requests data from the Backend API, which fetches it from the Database).
- **Hitting an API:** Sending an HTTP request to a specific API endpoint (URL).
- **API Architectures:** REST, GraphQL, SOAP, gRPC, WebSocket.
- **HTTP (HyperText Transfer Protocol):** The communication protocol between client and server.
- **HTTP vs HTTPS:**

- **HTTP:** Unencrypted data transfer.
- **HTTPS:** HTTP + TLS encryption. Secure and mandatory for production.

HTTP Request Anatomy

An HTTP request consists of: 1. **Method** (e.g., POST) 2. **URL/Path** (e.g., `/users`) 3. **Query Parameters** (e.g., `?sendEmail=true`) 4. **Headers** (e.g., `Content-Type: application/json`) 5. **Body** (e.g., JSON data)

Path Variable vs Query Parameter

- **Path Parameter:** Identifies a specific resource.
 - Example: `GET /users/10` (User ID 10)
 - Spring: `@PathVariable Long id`
- **Query Parameter:** Used for filtering, sorting, or pagination.
 - Example: `GET /users?page=1&size=10`
 - Spring: `@RequestParam int page`

HTTP Headers (Metadata)

- **Content-Type:** Format of the request/response body (e.g., `application/json`).
- **Accept:** Format the client expects in return.
- **Authorization:** Authentication credentials (e.g., `Bearer <token>`).

HTTP Methods

Method	Typical Purpose
GET	Retrieve data.
POST	Create a new resource.
PUT	Fully replace/update an existing resource.
PATCH	Partially update an existing resource.
DELETE	Delete a resource.

HTTP Status Codes

- **2xx (Success)**
 - `200 OK`: Request successful.
 - `201 Created`: Resource successfully created.
 - `204 No Content`: Request successful, no response body.
- **4xx (Client Error)**

- **400 Bad Request**: Invalid request/validation failed.
- **401 Unauthorized**: Missing or invalid authentication.
- **403 Forbidden**: Authenticated, but lacks permission.
- **404 Not Found**: Resource does not exist.
- **409 Conflict**: Request conflicts with current state.
- **422 Unprocessable Content**: Syntactically valid, semantic validation failed.
- **5xx (Server Error)**
 - **500 Internal Server Error**: Unexpected server-side crash.
 - **502 Bad Gateway**: Proxy received an invalid response.
 - **503 Service Unavailable**: Server overloaded or down.

2. Spring Boot REST API Implementation

Standard Layered Architecture

Controller → Service → Repository → Database

Key Dependencies

- **spring-boot-starter-web**: Provides Spring MVC, embedded Tomcat, Jackson (JSON).
- **spring-boot-starter-validation**: Provides validation annotations.

Serialization vs Deserialization (Parsing)

- **Serialization**: Converting a Java Object to JSON (Response).
- **Deserialization**: Converting incoming JSON to a Java Object (Request). Handled by Jackson in Spring.

DTO (Data Transfer Object)

Never expose DB Entities directly to the API. Use DTOs. * **Flow**: JSON Request → Request DTO → Service → Entity → DB → Entity → Response DTO → JSON Response.

Complete CRUD Controller Structure

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping
    public List<UserResponse> getAllUsers() { ... }
```

```

@GetMapping("/{id}")
public ResponseEntity<UserResponse> getUser(@PathVariable Long id) { ... }

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public UserResponse createUser(@Valid @RequestBody CreateUserRequest request) { .
.. }

@PutMapping("/{id}")
public UserResponse updateUser(@PathVariable Long id, @Valid @RequestBody
UpdateUserRequest request) { ... }

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Long id) { ... }
}

```

ResponseEntity

Used to fully control the HTTP response, including Status Code, Headers, and Body. (e.g., `ResponseEntity.status(HttpStatus.CREATED).body(user)`).

Validation & Exception Handling

- **Trigger:** Use `@Valid` next to `@RequestBody`.
- **Annotations:** `@NotNull`, `@NotBlank`, `@Email`, `@Size`, etc.
- **Global Exception Handler:** Avoid scattered `try-catch` blocks.

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(UserNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
}

```

3. Consuming External APIs & LLMs

RestClient (Spring Boot 3.2+)

Modern, synchronous HTTP client used to call external APIs (e.g., GitHub, Payment gateways, LLMs).

```
@Service
```

```

public class GithubClient {
    private final RestClient restClient = RestClient.create("https://api.github.com");

    public GithubUser getUser(String username) {
        return restClient.get()
            .uri("/users/{username}", username)
            .retrieve()
            .body(GithubUser.class); // Deserializes external JSON to Java Object
    }
}

```

LLM API Integration Architecture

Frontend → Spring Controller → Service → RestClient (LLM Client) → LLM Provider API (and back).

* **Note:** You must map the exact JSON schema provided by the LLM provider (e.g., `model`, `messages`, `role`) using Java DTOs.

4. REST vs GraphQL

REST Principles

- **Stateless:** Server doesn't store client session state between requests.
- **Resource-Oriented:** URLs represent resources (`/users`, `/orders`), not actions (`/getUsers`).
- **Uniform Interface:** Standard use of HTTP methods and status codes.

GraphQL Basics

A query language where the client specifies exactly what data it wants, solving over-fetching and under-fetching.

- **Operations:**
 1. **Query:** Read data (like GET).
 2. **Mutation:** Create/Update/Delete data (like POST/PUT/DELETE).
 3. **Subscription:** Real-time updates.

Spring Boot GraphQL Annotations

- `@Controller`: Marks the class.
- `@QueryMapping`: Maps a GraphQL Query.
- `@MutationMapping`: Maps a GraphQL Mutation.
- `@Argument`: Extracts parameters from the GraphQL query.

Comparison Table

Feature	REST	GraphQL
Endpoints	Multiple (one per resource)	Usually one (<code>/graphql</code>)
Response	Server defines the shape	Client requests specific fields
HTTP Methods	Crucial (GET, POST, PUT, DELETE)	Both GET, POST but POST is very common
Learning Curve	Simple, standard	Steeper, highly flexible

5. Spring Boot Annotations Cheat Sheet

- **Routing:** `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`
- **Request Data:** `@PathVariable`, `@RequestParam`, `@RequestBody`, `@RequestHeader`
- **Response:** `@ResponseBody`, `@ResponseStatus`, `ResponseEntity`
- **Validation:** `@Valid`, `@NotNull`, `@NotBlank`, `@Email`, `@Size`, `@Min`, `@Max`
- **Error Handling:** `@RestControllerAdvice`, `@ExceptionHandler`
- **GraphQL:** `@QueryMapping`, `@MutationMapping`, `@Argument`

6. End-to-End Mental Model (Request → Response)

1. **Client** sends HTTP Request.
2. **DispatcherServlet** intercepts it.
3. **Controller** receives it and validates DTO.
4. **Service** executes business logic.
5. **Repository** / **RestClient** fetches data from DB / External API.
6. **Service** maps Entity back to Response DTO.
7. **Jackson** serializes DTO to JSON.
8. **Client** receives HTTP Response.

7. Interview Prep Checklist & Questions

HTTP & API Concepts

Q: What is an API? REST API? Are REST and HTTP the same?

- **API:** A defined interface that allows two applications to communicate.
- **REST API:** An API built on REST architectural principles (stateless, resource-oriented).
- **REST vs HTTP:** They are not the same. HTTP is the protocol (the transport mechanism), while REST is the architectural style (the design rules for endpoints and data).

Q: Difference between GET vs POST? PUT vs PATCH?

- **GET vs POST:** GET fetches data without changing the database. POST creates new data and changes the database state.
- **PUT vs PATCH:** PUT replaces an entire resource completely. PATCH updates only specific fields of a resource.

Q: Path Variable vs Query Parameter?

- **Path Variable (/users/10):** Used to identify a specific, mandatory resource (User 10).
- **Query Parameter (/users?role=admin):** Used for optional filtering, sorting, or pagination.

Q: Content-Type vs Accept headers?

- **Content-Type:** Tells the receiver the format of the data being sent (e.g., "I am sending JSON").
- **Accept:** Tells the receiver what format the client expects back (e.g., "I want the response in JSON").

Q: Explain HTTP status codes: 200, 201, 204, 400, 401, 403, 404, 500.

- **200 OK:** Success.
- **201 Created:** Resource successfully created (POST).
- **204 No Content:** Success, but no data to return.
- **400 Bad Request:** Client sent invalid data.
- **401 Unauthorized:** Client is not authenticated (no valid login token).
- **403 Forbidden:** Authenticated, but lacks permission for this specific action.
- **404 Not Found:** Endpoint or resource does not exist.
- **500 Internal Server Error:** Backend code crashed.

Spring Boot REST

Q: Difference between @Controller and @RestController?

- **@Controller** is used to return HTML web pages.
- **@RestController** is used for APIs to return raw data (JSON). It automatically applies

`@ResponseBody` to all methods.

Q: `@PathVariable` vs `@RequestParam`?

- `@PathVariable` extracts data directly from the URI path (`@GetMapping("/users/{id}")`).
- `@RequestParam` extracts data from the query string after the `?` (`@GetMapping("/users")`).

Q: Why use `ResponseEntity`?

- It provides complete control over the HTTP response, allowing you to manually set the status code (e.g., 201 Created), custom headers, and the response body.

Q: What are DTOs and why use them?

- **Data Transfer Objects (DTOs)** are plain Java classes used to carry data between the client and server. They prevent exposing database Entities directly, hiding sensitive data like passwords and decoupling the database layer from the API layer.

Q: How does `@RestControllerAdvice` work?

- It acts as a global exception interceptor. Instead of writing `try-catch` blocks everywhere, it catches exceptions thrown anywhere in the application and translates them into clean, standardized JSON error responses.

Q: How to call an external API in Spring Boot? (`RestClient`)

- Use `RestClient` (introduced in Spring Boot 3.2). It is a modern, synchronous HTTP client used to send requests to external servers (like an LLM provider) and deserialize their JSON responses into Java objects.

GraphQL

Q: REST vs GraphQL?

- **REST:** Multiple endpoints, server dictates the response structure (prone to over/under-fetching).
- **GraphQL:** Single endpoint, client dictates exactly which fields it wants in the response.

Q: What are Queries, Mutations, and Subscriptions?

- **Query:** Fetches data (like GET).
- **Mutation:** Modifies data (like POST/PUT/DELETE).
- **Subscription:** Listens for real-time data updates via WebSockets.

Q: What is a GraphQL Schema?

- A strictly typed contract (usually a `.graphqls` file) defining all available queries, mutations, and data types so the client knows exactly how to interact with the API.